System Design Project Report

VECTRASLOT

Smart Parking Management System

Subject: System Design

Document Type: Project Architecture and Design Analysis

Project Analysis Document

April 28, 2026

Contents

1 Contribution Matrix **3**

2 Project Overview **3**

3 Problem Statement **3**

4 System Goals **4**
4.1 Functional Goals 4
4.2 Non-Functional Goals 4

5 High-Level Architecture **4**
5.1 Architectural Layers 5

6 Technology Stack **5**

7 Module Decomposition **5**
7.1 Authentication Module 5
7.2 Slot Module 6
7.3 Availability Module 6
7.4 Booking Module 6
7.5 Admin Module 6
7.6 Frontend Module 7

8 Data Design **7**
8.1 Core Entities 8
8.2 Relationships 8
8.3 Enumerations 9
8.4 Schema Design Remarks 9

9 UML Diagrams **9**
9.1 Sequence Diagram 10
9.2 Class Diagram 11

10 Applied SOLID Principles **11**
10.1 Single Responsibility Principle 11
10.2 Open/Closed Principle 12
10.3 Dependency Inversion Principle 12

11 Applied Design Patterns **12**
11.1 Layered Architecture 12
11.2 Strategy Pattern 12
11.3 Constructor Injection 13
11.4 Shared Client Pattern 13

12 Request Flow and Control Flow **13**
12.1 Authentication Flow 13
12.2 Availability Check Flow 13
12.3 Booking Creation Flow 14
12.4 Admin Dashboard Flow 14

13 API Design Summary **14**
13.1 Public and Auth APIs 14
13.2 Authenticated Booking APIs 14
13.3 Admin APIs 15

14 Security Design **15**
14.1 Authentication and Authorization 15
14.2 Credential Protection 15
14.3 Validation 15
14.4 Security Limitations 15

15 Scalability and Reliability Discussion **16**
15.1 Current Strengths 16
15.2 Potential Bottlenecks 16
15.3 Recommended Future Improvements 16

16 Strengths and Current Gaps **17**
16.1 Strengths 17
16.2 Current Gaps 17

17 Conclusion **17**

1 Contribution Matrix

S. No.	Name	Enrollment Number	Contribution
1	Guru Manohar Gupta	2401010125	Backend,Report,Diagrams
2	S Harsith Priyan	2401010398	Backend, Architecture Planning, Database
3	Deepanshu Chaudhary	2401010145	Backend, FrontEnd
4	Hemamdharnath Dakarapu	2401010137 4	Frontend, Project Architecture
5	JayaDeep	2401010172	Frontend, Diagrams

2 Project Overview

VectraSlot is a full-stack parking slot reservation and management system. It allows end users to register, authenticate, check slot availability for a time window, create bookings, and manage their reservations. It also provides a separate administrative interface for managing users, parking slots, bookings, and high-level operational statistics.

From a system design perspective, the project follows a modular client-server architecture with three clear layers:

- A React frontend for user and admin interaction.
- An Express and TypeScript backend that exposes REST APIs.
- A PostgreSQL database accessed through Prisma ORM.

The project demonstrates key system design concepts such as layered architecture, role-based access control, transactional booking logic, modular APIs, and a normalized relational schema.

3 Problem Statement

Traditional parking operations often suffer from manual allocation, unclear availability, and poor visibility into active reservations. VectraSlot addresses this by providing a centralized platform where:

- users can discover available parking slots for a selected time range,
- bookings can be created without double allocation,
- administrators can monitor usage and maintain system data,
- access to sensitive operations is controlled through roles.

4 System Goals

4.1 Functional Goals

- Register and authenticate users securely.
- Support separate login behavior for `USER` and `ADMIN`.
- View public slot inventory and time-based slot availability.
- Create, update, cancel, and complete bookings.
- Allow administrators to manage users, slots, bookings, and dashboard metrics.

4.2 Non-Functional Goals

- Maintain modular and readable code organization.
- Prevent booking conflicts for overlapping time ranges.
- Keep role enforcement simple and explicit.
- Support future scaling through clear module boundaries.

5 High-Level Architecture

The project follows a layered web application architecture. The frontend communicates with backend REST endpoints over HTTP, while the backend delegates persistence to Prisma, which in turn interacts with PostgreSQL.

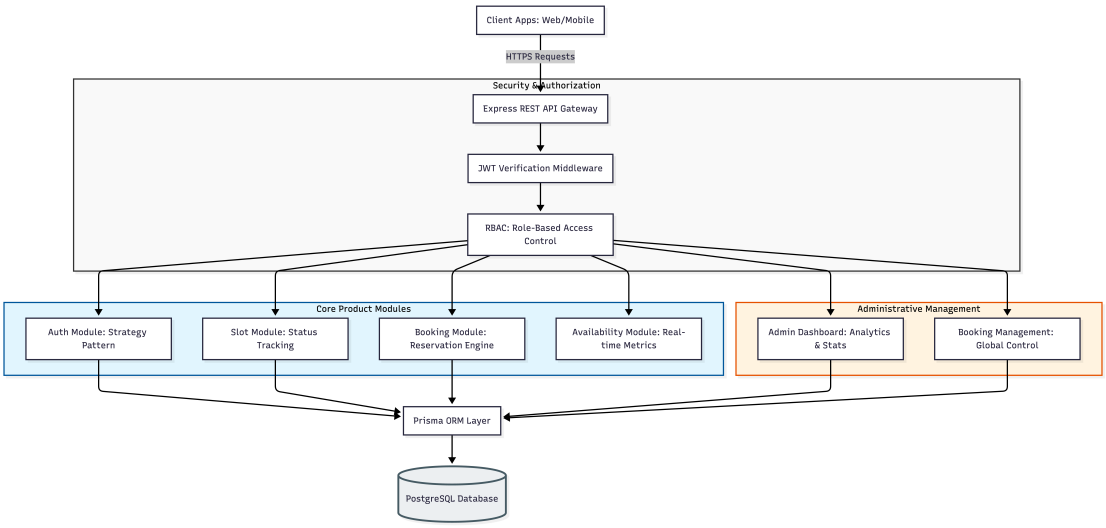

Figure 1: High-level project architecture of VectraSlot

5.1 Architectural Layers

Layer	Responsibility
Frontend	Presents user and admin dashboards, handles route protection, manages forms, and invokes REST APIs.
Backend API	Accepts requests, validates input, enforces authorization, applies business rules, and returns structured responses.
Service Layer	Encapsulates domain logic such as slot creation, booking conflict detection, role-aware authentication, and admin analytics.
Data Access Layer	Uses Prisma client for persistence, querying, transactions, and relational mapping.
Database	Stores normalized entities such as users, parking slots, and bookings.

6 Technology Stack

Category	Technology
Frontend	React, TypeScript, React Router, Vite, Tailwind CSS
Backend	Node.js, Express, TypeScript
Database Access	Prisma ORM with PostgreSQL adapter
Database	PostgreSQL
Authentication	JWT, bcryptjs
Validation	Zod
Build and Tooling	TypeScript compiler, nodemon, ts-node

7 Module Decomposition

The backend is divided into feature-based modules. Each module generally follows a route-controller-service split, which improves clarity and keeps the request lifecycle easy to trace.

7.1 Authentication Module

The authentication module handles registration and login. It uses a strategy-based design to differentiate between `USER` and `ADMIN` login flows.

Key responsibilities:

- user registration,
- password hashing,
- JWT token generation,
- admin secret validation,

- role-specific login policy enforcement.

7.2 Slot Module

The slot module manages parking slot inventory.

Key responsibilities:

- create a new slot,
- list all slots,
- update slot number or status.

7.3 Availability Module

The availability module computes which slots are available for a given time interval.

Core logic:

1. validate the requested `startTime` and `endTime`,
2. find active bookings that overlap the requested window,
3. collect conflicting slot identifiers,
4. return slots that are marked `AVAILABLE` and are not part of the conflict set.

7.4 Booking Module

The booking module is the core business domain of the system.

Key responsibilities:

- create new bookings,
- list bookings for the authenticated user,
- retrieve a booking by id,
- update booking time range,
- cancel bookings,
- complete bookings.

7.5 Admin Module

The admin module aggregates operational capabilities into one protected domain.

Key responsibilities:

- view and manage users,
- create, update, and delete slots,
- view and manage all bookings,
- compute system-level statistics.

7.6 Frontend Module

The frontend is organized around user journeys rather than low-level data access. It includes:

- landing page,
- login page,
- signup page,
- admin dashboard,
- user dashboard,
- API utility layer,
- local session storage utilities.

8 Data Design

VectraSlot uses a relational schema centered on three main entities: `User`, `ParkingSlot`, and `Booking`.

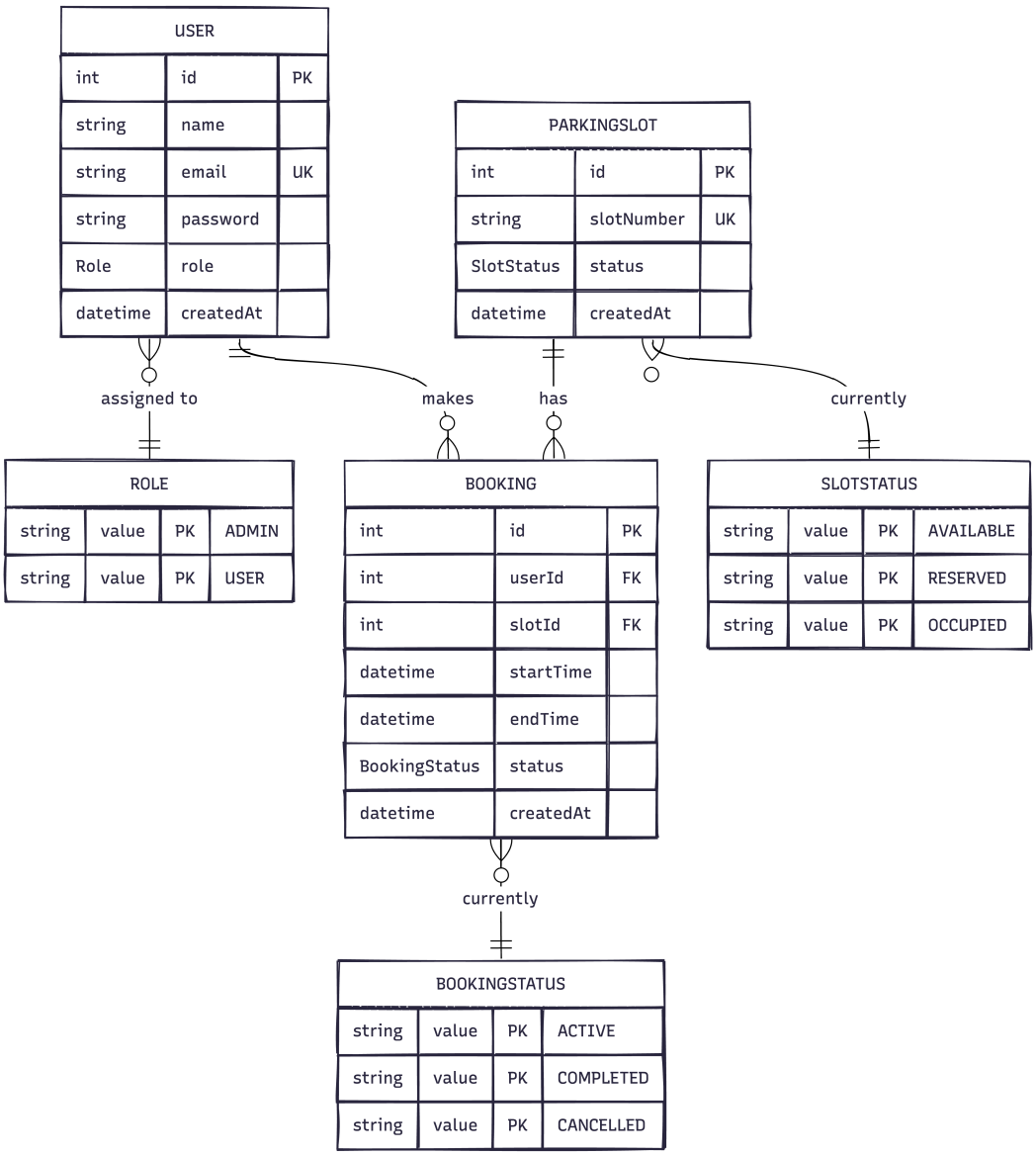

Figure 2: Entity relationship view of the VectraSlot data model

8.1 Core Entities

Entity	Purpose
User	Stores account identity, hashed password, and role.
ParkingSlot	Represents a physical parking slot with unique slot number and operational status.
Booking	Represents a reservation made by a user for a slot during a time window.

8.2 Relationships

- One user can have many bookings.

- One parking slot can have many bookings over time.
- Each booking belongs to exactly one user and one parking slot.

8.3 Enumerations

The schema uses enums to reduce ambiguity:

- `Role`: `ADMIN`, `USER`
- `SlotStatus`: `AVAILABLE`, `RESERVED`, `OCCUPIED`
- `BookingStatus`: `ACTIVE`, `COMPLETED`, `CANCELLED`

8.4 Schema Design Remarks

- Unique email and unique slot number constraints prevent duplicate identity and inventory records.
- Indexes on `Booking.userId` and `Booking.slotId` support common lookup paths.
- Time-window conflict detection is handled at the application layer rather than through a specialized database exclusion constraint.

9 UML Diagrams

This section is reserved for the primary UML artifacts associated with the system design of VectraSlot.

9.1 Sequence Diagram

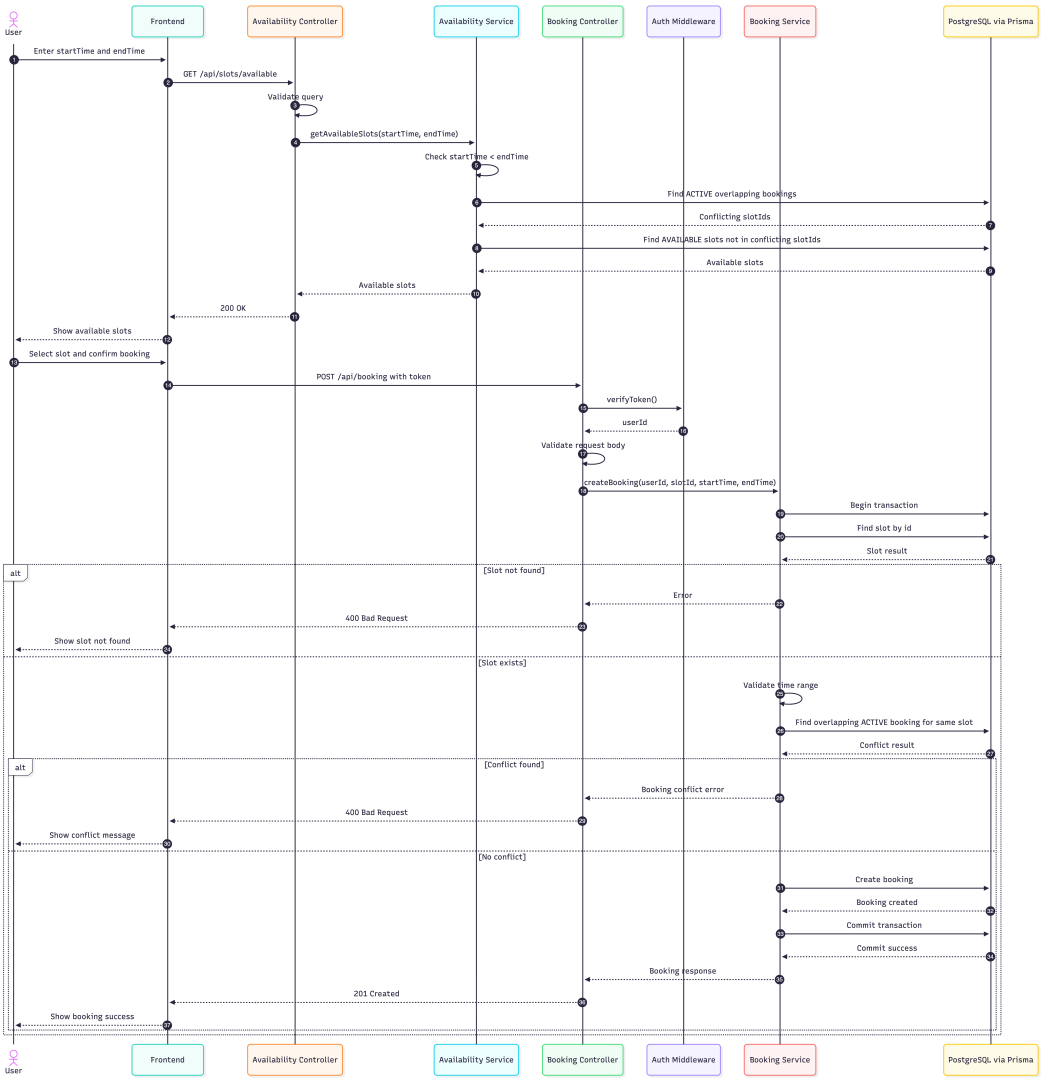

Figure 3: Sequence diagram of the booking interaction flow

9.2 Class Diagram

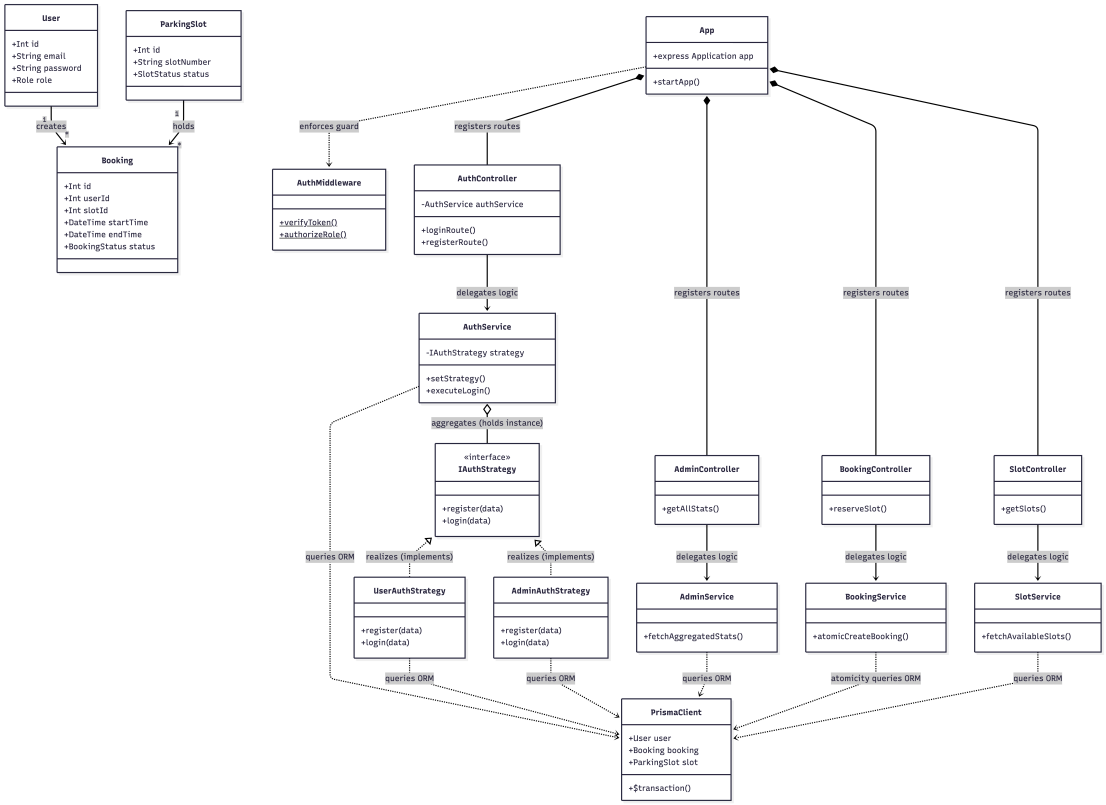

Figure 4: UML class diagram of the VectraSlot system

10 Applied SOLID Principles

Only the SOLID principles that are clearly reflected in the present codebase are discussed below.

10.1 Single Responsibility Principle

The project applies Single Responsibility Principle consistently across backend modules. Each layer has a focused purpose:

- routes define endpoint mappings,
- controllers handle HTTP request and response flow,
- services contain business rules,
- validation files define input contracts,
- middleware enforces authentication and authorization.

This separation is visible in modules such as `auth`, `booking`, `slots`, `availability`, and `admin`. For example, booking conflict logic is placed in `booking.service.ts`, while request parsing remains in `booking.controller.ts`. This improves maintainability because changes in business rules do not require controller rewrites.

10.2 Open/Closed Principle

The authentication design partially reflects the Open/Closed Principle. The `AuthService` depends on the `IAuthStrategy` abstraction, and concrete behaviors are provided by `UserAuthStrategy` and `AdminAuthStrategy`. As a result, authentication behavior can be extended by adding another strategy class without redesigning the service contract itself.

This is important from a system design perspective because role-specific policy is isolated behind a stable interface rather than mixed into one growing conditional block.

10.3 Dependency Inversion Principle

Dependency Inversion is applied in the slot module where `SlotController` receives `SlotService` through constructor injection. The controller depends on a service abstraction boundary instead of embedding data access logic directly.

While the project does not use a full dependency injection container, this pattern still improves testability and reduces coupling between request handling and business logic.

11 Applied Design Patterns

Only the design patterns that are directly implemented in the code are included here.

11.1 Layered Architecture

The strongest architectural pattern in the project is layered architecture. Each request moves through clearly separated stages:

1. route,
2. controller,
3. service,
4. Prisma data access,
5. database.

This structure is consistently followed across the major backend modules. It improves readability, localizes change, and makes the system easier to extend.

11.2 Strategy Pattern

The authentication module implements the Strategy Pattern through:

- `IAuthStrategy`,
- `UserAuthStrategy`,
- `AdminAuthStrategy`,
- `AuthService` as the context that switches strategies.

This pattern is appropriate because admin and user login rules are not identical. Admin login requires an additional secret and stricter privilege validation. By encapsulating those rules in separate strategy classes, the code remains structured and avoids scattered role-based branching.

11.3 Constructor Injection

Constructor injection is applied in the slot module where the controller accepts its service dependency through the constructor. This is a small but meaningful design pattern because it separates object creation from object usage and supports cleaner modular structure.

11.4 Shared Client Pattern

The Prisma client is created in a central utility module and reused across services. This follows a shared client approach similar to a singleton-style access pattern for infrastructure resources. It ensures that database access configuration remains centralized and consistent throughout the application.

12 Request Flow and Control Flow

12.1 Authentication Flow

1. Client submits login credentials with a declared role.
2. Controller resolves the appropriate strategy.
3. Strategy validates account existence and credentials.
4. Backend generates JWT and returns authenticated user metadata.
5. Frontend stores token and user details for route-based access.

12.2 Availability Check Flow

1. User selects `startTime` and `endTime`.
2. Frontend calls `/api/slots/available`.
3. Backend validates query parameters.
4. Service finds overlapping active bookings.
5. Service returns only slots that are both operationally available and not reserved in that window.

12.3 Booking Creation Flow

1. Authenticated user selects a slot and time window.
2. Booking endpoint verifies JWT and extracts user identity.
3. Service starts a database transaction.
4. Slot existence and time validity are checked.
5. Overlapping active bookings for the same slot are searched.
6. If no conflict exists, a new booking is created and committed.

12.4 Admin Dashboard Flow

1. Admin logs in with password and admin secret.
2. Frontend loads users, slots, bookings, and statistics in parallel.
3. Backend verifies token and admin role for every admin endpoint.
4. Admin mutations trigger API updates followed by dashboard refresh.

13 API Design Summary

13.1 Public and Auth APIs

Method	Endpoint	Purpose
POST	`/api/auth/register`	Create user account
POST	`/api/auth/login`	Login for user or admin
GET	`/api/slots`	List slot inventory
GET	`/api/slots/available`	Query time-window-based availability

13.2 Authenticated Booking APIs

Method	Endpoint	Purpose
POST	`/api/booking`	Create booking
GET	`/api/booking/my`	List current user's bookings
GET	`/api/booking/:id`	Retrieve booking by id
PATCH	`/api/booking/:id`	Update booking time range
PATCH	`/api/booking/:id/complete`	Mark booking as completed
DELETE	`/api/booking/:id`	Cancel booking

13.3 Admin APIs

Method	Endpoint	Purpose
GET	`/api/admin/users`	View users
PATCH	`/api/admin/users/:id/role`	Change user role
DELETE	`/api/admin/users/:id`	Delete user
POST	`/api/admin/slots`	Create slot
GET	`/api/admin/slots`	View all slots with active bookings
PATCH	`/api/admin/slots/:id`	Update slot
DELETE	`/api/admin/slots/:id`	Delete slot
GET	`/api/admin/bookings`	View all bookings
PATCH	`/api/admin/bookings/:id`	Modify booking
DELETE	`/api/admin/bookings/:id`	Delete booking
GET	`/api/admin/stats`	View system statistics

14 Security Design

14.1 Authentication and Authorization

The system uses JWT-based stateless authentication. Tokens are issued during login and sent by the client in the `Authorization` header. The backend middleware verifies the token and attaches user identity to the request.

Authorization is role-based:

- regular users can access booking operations associated with their accounts,
- admins can access management and analytics endpoints,
- admin login additionally requires an `ADMIN_SECRET`.

14.2 Credential Protection

Passwords are hashed using `bcryptjs`. This prevents plain text password storage and supports safe credential verification.

14.3 Validation

The codebase uses Zod schemas in several modules for request validation. This improves reliability by rejecting malformed input before business logic runs.

14.4 Security Limitations

The current design is solid for a project setting, but some production-grade enhancements would still be valuable:

- stronger global validation consistency across all routes,

- centralized error handling middleware,
- token refresh or revocation strategy,
- rate limiting for authentication endpoints,
- stricter environment management for secrets and configuration.

15 Scalability and Reliability Discussion

15.1 Current Strengths

- Transaction-based booking logic improves consistency.
- Indexed foreign keys support common read operations.
- Stateless JWT authentication simplifies horizontal scaling of the backend.
- Clear module boundaries support maintainability.

15.2 Potential Bottlenecks

- availability queries depend on scanning overlapping bookings and may grow costlier as booking volume increases,
- polling every few seconds from dashboards can increase request load,
- admin endpoints aggregate multiple data domains and may become heavy with larger datasets,
- conflict checks currently rely on application logic rather than a stronger database-level concurrency guard.

15.3 Recommended Future Improvements

- add pagination and filtering for admin listings,
- introduce centralized logging and monitoring,
- normalize error response format across all endpoints,
- add database constraints or locking strategies for stronger concurrent booking guarantees,
- consider caching or materialized summaries for dashboard statistics,
- move from polling to push-based updates if real-time scale becomes important.

16 Strengths and Current Gaps

16.1 Strengths

- Clean full-stack separation with understandable module boundaries.
- Meaningful role-based architecture rather than only cosmetic role labeling.
- Transactional booking workflow for conflict-sensitive logic.
- Relational schema that matches the domain naturally.
- Frontend and backend naming are mostly aligned around core business concepts.

16.2 Current Gaps

- public slot mutation endpoints exist without enforced admin middleware,
- some response shapes differ across modules,
- error handling is distributed rather than centralized,
- some update operations accept broad request bodies and could use stricter validation,
- route naming mixes singular and plural conventions in a few places.

17 Conclusion

VectraSlot is a structured parking reservation system with clear architectural boundaries between interface, application logic, and persistence. The codebase demonstrates applied system design concepts through modular services, role-based access control, transactional booking handling, and a well-aligned relational schema.

Its strongest technical qualities are the layered backend structure, the strategy-based authentication model, and the conflict-aware booking workflow. These choices make the project suitable for system design evaluation while still remaining practical and understandable.